Fingertip Part Recognition

A Vision-AI + Hand-Tracking Experiment

Build & Troubleshooting Documentation — From Concept to Working System

Prepared: 28 July 2026

Project: raakesh-test-13052026 (Vertex AI / GCP)

Contents

1. Project Overview

The goal of this project was to determine whether a Vision AI model, combined with real-time hand tracking, can identify the specific PART of an object that a user's index finger is pointing at — not merely classify the whole object. For example, pointing at a lion should return "lion's eye," not just "lion."

The system was designed around a three-stage architecture, using a dedicated tool for each job rather than asking one model to do everything:

- **Stage 1 — Fingertip localisation:** MediaPipe Hands finds the exact index-fingertip pixel, locally, at 30fps.
- **Stage 2 — Dwell gate:** Decides WHEN to ask the vision model — only when the finger holds still — so calls are deliberate, not continuous.
- **Stage 3 — Vision AI:** Names the object and the part beneath a marked point, returning OBJECT | PART | DETAIL.

Two hard problems were designed around from the start: **occlusion** (the finger covers the part it points at, solved by sampling slightly ahead of the fingertip and keeping a "clean plate" frame from before the hand entered) and **timing** (the vision call is slow relative to video, solved by running it on a background thread gated on dwell).

2. The Central Challenge: Getting a Working Vision Model

The vision pipeline itself worked early. The overwhelming majority of the effort went into one problem: obtaining a vision model that would actually respond. What looked like a series of unrelated errors turned out to be a single underlying reality — free-tier hosted vision was effectively unavailable for this setup (India, no billing), and every hosted free path ultimately routed back to the same saturated or zero-quota Google infrastructure.

2.1 The 8 hosted free-tier models tested (all failed)

Eight models were tried across two providers and two Google accounts before abandoning the hosted free-tier approach:

#	Model	Provider	Result
1	gemini-flash-latest	Google	404 — invalid model name
2	gemini-2.0-flash	Google	Worked briefly, then quota exhausted
3	gemini-2.5-flash	Google	Quota exhausted (limit: 0)
4	gemini-2.0-flash-lite	Google	Quota exhausted (limit: 0)
5	gemini-2.0-flash (2nd account)	Google	Quota exhausted (limit: 0)
6	google/gemma-4-31b-it:free	OpenRouter	429 — rate-limited upstream
7	google/gemma-4-26b-a4b-it:free	OpenRouter	429 — rate-limited upstream
8	nvidia/nemotron-3-nano-omni:free	OpenRouter	429 — rate-limited upstream

Key insight: the OpenRouter free models all reported the upstream provider as "Google AI Studio" — the same free pool that returned limit: 0 directly. Switching models could not help because they shared one congested upstream. Creating a second Google account did not help either: the free-tier quota provisioned at zero regardless, a consequence of Google's late-2025 free-tier reductions.

2.2 The 9th model — local, and the turning point

Model	Provider	Result
moondream (via Ollama)	Local	Ran with no limits, but accuracy too low for the study

Moondream proved the pipeline could run with zero quota dependence, but the laptop (Intel i5-6300U, integrated graphics, 8 GB RAM) ran it CPU-only at 10–30 seconds per answer, and its accuracy on fine parts was weak. It served as a fallback, not the final answer.

3. The API-Key Dead End

With free tiers exhausted, the plan shifted to using a company GCP credit (a $25,000 Ecosystem Partner grant) via Vertex AI. The instinct was to "create an API key" — the mental model carried over from the consumer Gemini API. This led down a second series of blocked attempts, all rooted in one fact: the organisation had disabled service-account keys by policy.

Attempt	Outcome
JSON service-account key	BLOCKED — org policy iam.disableServiceAccountKeyCreation
Service-account-bound API key	BLOCKED — 'Agent Platform API' greyed out until bound; binding disabled
'Authenticate through service account' option	NOT AVAILABLE — hidden/disabled by the same policy
Reusing old consumer Gemini key	WRONG PATH — routed to depleted AI Studio prepay, not GCP credit

The realisation that unlocked everything: Vertex AI does not use an API key at all. "Use GCP credits to create an API key" combines two unrelated things — credits pay for usage; a key (or a login) authenticates. The credit-spending path (Vertex) authenticates via a Google Cloud login, and the API-key path (AI Studio) does not touch GCP credits. They are opposite roads.

4. The Solution That Worked

The working path used Vertex AI (rebranded mid-2026 as the Gemini Enterprise Agent Platform), authenticated by Application Default Credentials from a local gcloud login — no key anywhere — billed to the company GCP credit.

4.1 Setup sequence

1. Confirmed the GCP credit and identified the project (raakesh-test-13052026).
2. Enabled the Agent Platform API (aiplatform.googleapis.com) on the project.
3. Installed the Google Cloud CLI (gcloud).
4. Authenticated with: `gcloud auth application-default login --no-launch-browser`
5. Installed the SDK: `pip install google-genai`
6. Created the client in Vertex mode: `genai.Client(vertexai=True, project=..., location=...)`

Working model: `gemini-2.5-flash-lite` in region `us-central1`. (gemini-3.1-flash-lite returned 404 — not available in that region under that name; 2.5-flash-lite was.)

4.2 First successful result

```
circle | center | The center is the point equidistant from all points on the circumference.
```

Then, live from the webcam: `face | cheek | The cheek is the fleshy area between the eye and the mouth.` — object, exact part, and factual detail, in 2.85 seconds, on the company credit.

5. Technical Issues Solved Along the Way

Beyond the model-access saga, a series of concrete code and environment errors were diagnosed and fixed:

Symptom	Cause	Fix
ModuleNotFoundError: mediapipe	Wrong Python / no wheel	Installed into Python 3.11 (has the wheel)
module 'mediapipe' has no attribute 'solutions'	Legacy Solutions API removed	Wrote hand_tracker.py adapter using the Tasks API
Python 3.14 — mediapipe unavailable	No wheel for 3.14	Pinned project to Python 3.11
venv Permission denied / locked	Antivirus / running process	Ran directly with py -3.11
Detail truncated ('The', 'L')	'Thinking' tokens ate the budget	Disabled thinking (thinking_budget = 0)
'wall | surface' misfires	Point-ahead offset overshoot	Reduced offset; added live [ ] tuning + debug view
Repeated wrong object	Stale clean-plate reused	Reject clean plate older than 3s; fall back to live frame
HTTP 400 on Gemini	Wrong / retired model name	Switched to a current model name
gcloud login hung / browser 400	Stale browser session / handoff	Used --no-launch-browser code-paste flow
Vertex 404 not found	Model not in region	Used gemini-2.5-flash-lite in us-central1

6. Final Working Architecture

- **Local (free, on the laptop):** Webcam (OpenCV) → MediaPipe hand tracking (hand_tracker.py) → fingertip + point-ahead geometry → dwell gate → marked, cropped query image.
- **Cloud (GCP credit):** vertex_vision.py sends the image to Gemini 2.5 Flash-Lite via Vertex AI, authenticated by the gcloud login, and returns OBJECT | PART | DETAIL.
- **Logging:** Every prediction written to experiment_log.csv with mode, latency, target coordinates, clean-plate flag, and a manual correct/incorrect mark (y/n).
- **Experiment design:** Mode A (VLM alone) vs Mode B (hand-tracking pipeline). The accuracy gap between them is the core finding.

7. Key Lessons

7. **Free hosted vision is a moving target.** Quotas, model names, and shared-pool capacity change constantly; three separate free routes each walled the project.
8. **Read the error, not the assumption.** A 429 'limit: 0' is a quota state, not a deprecated model — this distinction saved chasing false fixes.
9. **Credits and keys are unrelated.** The credit-spending path (Vertex) uses a login, not a key; the key path doesn't touch credits.

10. **Org policy is a feature, not a bug.** Disabled service-account keys forced the more secure gcloud-login method — which was also simpler.
11. **Separate localisation from semantics.** Hand tracking finds the point; the VLM only names it. Neither has to do the other's job.
12. **Isolate to diagnose.** A tiny no-camera test script (test_gemini / vertex_vision main) repeatedly pinpointed whether a failure was the key, the model, the quota, or the code.

End of document — system verified working on 28 July 2026.